

Sesión 8: Inteligencia Artificial como Copiloto

El Desarrollador al Mando

.NET 10 / CQRS

ANGULAR 22 / SIGNALS

CLEAN ARCHITECTURE

AGENTIC SDD PIPELINE

Proyecto: Migración de 9 Sistemas Legacy (GeneXus ACL → Arquitectura Hexagonal)

Presentador: Lead Software Engineer & Principal AI Architect

El Desarrollador al Mando: Principio Fundamental

Aceleración de la Migración sin Sacrificar la Arquitectura ni la Calidad

LA REGLA DE ORO

"Comprender los patrones primero; guiar a la IA después."

La Inteligencia Artificial magnifica exponencialmente la velocidad de ejecución y la generación de código base, pero el **desarrollador humano** mantiene el 100% de la **responsabilidad** sobre la arquitectura, la encapsulación del dominio y los estándares de calidad.

EL RETO DE MIGRACIÓN

- **9 Sistemas Legacy:** Transición de esquemas tabulares GeneXus a Bounded Contexts.
- **Cero Deuda Técnica Acumulada:** Prohibido copiar anti-patrones legacy a la nueva arquitectura.
- **Copiloto vs. Piloto Autónomo:** La IA propone, el desarrollador evalúa, refina y valida antes de integrar.

Cultura General: De la Inteligencia Artificial al LLM

Desmitificando los Conceptos Básicos del Procesamiento de Lenguaje

1. IA & MACHINE LEARNING

De Reglas a Modelos Estadísticos

La Inteligencia Artificial abarca desde sistemas basados en reglas hasta algoritmos de **Machine Learning** que aprenden patrones directamente a partir de datos masivos sin ser programados explícitamente.

2. LLMS (LARGE LANGUAGE MODELS)

Arquitectura Transformer

Un LLM es una red neuronal profunda basada en mecanismos de *Atención* (Transformers) entrenada con petabytes de texto para calcular la probabilidad del lenguaje humano.

3. TOKENS & EMBEDDINGS

La Matemática del Texto

El modelo no lee palabras; convierte el texto en **Tokens** (~0.75 palabras) y **Embeddings** (vectores numéricos de 1,536+ dimensiones) para procesar relaciones de significado.



Dato Clave: Un LLM no "piensa" en conceptos abstractos; procesa números y probabilidades matemáticas a velocidad hiperbólica.

La Farsa de la "Memoria": Naturaleza Stateless de las APIs de IA

Por qué el Modelo NO Recuerda Quién Eres ni Qué Dijiste Hace 5 Minutos

LA ILUSIÓN DE LA MEMORIA

La Interfaz de Chat vs. La API Real

- **El Mito de la Continuidad:** Las aplicaciones web de chat hacen parecer que el modelo "recuerda" la conversación.
- **La Realidad Técnica:** Los endpoints de las APIs de LLM son **100% Stateless** (HTTP POST sin estado de sesión persistente en

MECANISMO TÉCNICO

Payload Acumulativo & Implicaciones

- **Re-Envío Completo:** Para mantener contexto, la aplicación cliente DEBE reenviar todo el historial acumulado en cada turno de conversación:

```
messages: [  
  { role: 'system', content: ' ... ' },
```

Fundamentos: La Naturaleza Estocástica de los LLMs

Predicción de Tokens vs. Ejecución Determinista de Código

¿Cómo Funciona un LLM?

- **Modelo Probabilístico:** Selecciona el siguiente token basado en distribuciones de probabilidad ($P(W_n | W_1 \dots W_{n-1})$).
- **Temperatura y Muestreo:** Introduce variabilidad intencional; el mismo prompt puede producir respuestas diferentes.
- **Sin Motor de Ejecución Intrínseco:** Un LLM *no ejecuta código mentalmente*; simula patrones de texto aprendidos durante el entrenamiento.

Código Tradicional vs. LLM

Dimensión	Compilador / CLI	LLM (Copiloto)
Comportamiento	Determinista (100%)	Estocástico
Entrada igual	Misma Salida	Salidas variables
Validación	Sintaxis estática	Empírica y contrato

⚠ **Alerta:** La fluidez sintáctica del texto generado por la IA crea una falsa sensación de certeza. ¡Verifica siempre!

La Ventana de Contexto y el Fenómeno "Lost in the Middle"

Atención Limitada en Modelos de Lenguaje Masivos

1. CAPACIDAD DE CONTEXTO

El Límite Físico de Tokens

Aunque los modelos modernos ofrecen ventanas de 128k a 1M+ de tokens, enviar codebases enteros degrada drásticamente la capacidad de razonamiento.

2. DEGRADACIÓN DE ATENCIÓN

"Lost in the Middle"

Los Transformers prestan alta atención al inicio y al final, pero **ignoran detalles críticos** ubicados en el centro del payload.

3. ESTRATEGIA DE MITIGACIÓN

Hidratación JIT y Re-Fetch

No alimentes al LLM con todo el proyecto. Pasa referencias cortas (Claim-Check) e hidrata contexto únicamente en la tarea activa.



Regla de Arquitectura: Menos es más. Un prompt de 2,000 tokens altamente enfocado genera código superior a un prompt saturado de 50,000 tokens.

Del Prompting Caótico a la Automatización Determinista

Reemplazando Prompts Volátiles por Flujos Estandarizados

El Antipatrón: Chat Web & Prompts Ad-Hoc

- Prompts improvisados en chats web individuales.
- Respuestas inconsistentes que varían por desarrollador.
- Sin versionamiento en Git ni control de calidad.
- Fuga de reglas de arquitectura y patrones del proyecto.

La Solución Enterprise: Pipelines Agénticos

- **Scripts & CLIs:** Automatización gestionada mediante comandos (`python script.py`).
- **Plantillas Versadas:** Prompts de agentes y skills almacenados en el repositorio (`prompts/agent.yaml`).
- **Validación de Contratos:** Linters, compilación estricta y tests automatizados

Componentes Esenciales del Ecosistema de IA

Definiendo la Infraestructura de un Sistema Agéntico Enterprise



1. Agente (Agent)

Entidad autónoma con rol definido (ej. `strategic-ddd-architect`), ciclo de razonamiento (Plan → Execute → Verify) y consumo de herramientas.



2. Skill / Tool

Capacidades mecánicas invocables (ej. leer un archivo específico, consultar la lista blanca de dependencias, ejecutar un linter o compilar un proyecto).



3. Knowledge (Base de Conocimiento)

Documentación técnica, esquemas de BD y guías de arquitectura (ej. `tech-stack-whitelist.md`) que proveen contexto estático verificado.



4. Rules / System Instructions

Restricciones infranqueables e invariantes de negocio (ej. Mandato de Cero Alucinación, Tablas GeneXus Read-Only) que el modelo DEBE acatar strictly.

Prompts Contextuales Estandarizados

La Fórmula de 5 Elementos para la Generación de Código Limpio

Prompt Efectivo = Contexto + Rol + Tarea + Restricciones + Ejemplos (Few-Shot)

CONTEXTO

Especificación del sistema legacy, esquema de base de datos actual y tecnologías objetivo (.NET 10 / Angular 22).

ROL

"Actúa como un Lead Backend Architect experto en DDD, CQRS y MediatR en .NET 10."

TAREA

"Genera el Command Handler para la aprobación de préstamos fideicomitados respetando inmutabilidad."

RESTRICCIONES

"Prohibido usar EF Core en capa de dominio. Prohibido setters públicos. Retorna siempre `Result<T>`."

FEW-SHOT

Proporciona un fragmento de código de ejemplo que ilustre la estructura de carpetas y nombrado estándar del proyecto.

Caso Práctico Backend: CQRS & MediatR Handlers

Comparativa: Prompt Ad-Hoc vs. Prompt Contextual Estructurado

❌ 1. PROMPT AD-HOC (GENERA CÓDIGO ACOPLADO)

ANTIPATRÓN

Prompt: "Hazme un handler en C# para aprobar préstamos."

```
1  public class AprobarPrestamoHandler {
2      private readonly AppDbContext _db;
3      public AprobarPrestamoHandler(AppDbContext db) => _db = db;
4
5      public async Task Handle(AprobarCmd cmd) {
6          var p = await _db.Prestamos.FindAsync(cmd.Id);
7          p.Estado = "APROBADO"; // Setter abierto, mutación directa de estado!
8          await _db.SaveChangesAsync(); // Invocación directa a EF Core dentro del Handler!
9      }
10 }
```

✅ 2. PROMPT CONTEXTUAL ESTRUCTURADO (CLEAN ARCHITECTURE)

PATRÓN CORRECTO

Prompt: Con Pol strategic-ddd-architect, Constraints & Result pattern

Caso Práctico Dominio: Entidades y Value Objects

Protegiendo las Reglas de Negocio del Sistema Legacy

✗ 1. DOMINIO ANÉMICO (ALUCINACIÓN COMÚN)

SIN INVARIANTES

```
1 public class Prestamo {
2     public Guid Id { get; set; }
3     public decimal Monto { get; set; }
4     public string Estado { get; set; } // Mutable libremente desde cualquier lugar!
5     public DateTime FechaCreacion { get; set; }
6 }
```

⚠ Defecto: El modelo permite instanciar préstamos en estados inválidos (ej. Monto negativo), violando la encapsulación.

✓ 2. RICH DOMAIN MODEL (PROMPT ESTANDARIZADO)

RICH DOMAIN

```
1 public class Prestamo : AggregateRoot {
2     public LoanId Id { get; private set; }
3     public Money Monto { get; private set; }
4     public LoanStatus Estado { get; private set; }
5
6     private Prestamo() { } // Constructor privado para EF Core
```

Caso Práctico Frontend: Angular 22 & Nx Architecture

Garantizando Reactividad Fina sin Acoplamiento HTTP

❌ 1. COMPONENTE ANGULAR LEGACY / ACOPLADO

ACOPLADO A HTTP

```
1  @Component({ selector: 'app-loan', template: `...` })
2  export class LoanComponent implements OnInit {
3      loans: any[] = [];
4      constructor(private http: HttpClient) {} // HTTP directo en UI Component!
5
6      ngOnInit() {
7          this.http.get('/api/loans').subscribe(res => {
8              this.loans = res as any[]; // Memory leak por falta de unsubscribe & tipo any!
9          });
10     }
11 }
```

✅ 2. COMPONENTE ANGULAR MODERNO (SIGNALS & CLEAN UI)

ANGULAR 22 SIGNALS

Evaluación Crítica y Detección de Alucinaciones

Identificando Vicios Comunes en el Código Generado por IA

 **1. Violaciones de Encapsulamiento**

La IA tiende por defecto a generar miembros `public` y setters abiertos para satisfacer pruebas rápidamente. **Acción:** Exigir constructores privados y factory methods explícitos.

 **2. Fuga de Infraestructura a Dominio**

Inclusión desapercibida de namespaces de ORM (`using Microsoft.EntityFrameworkCore`) o paquetes de cliente HTTP en proyectos de `Core.Domain`. **Acción:** Validar imports mediante linter de arquitectura.

 **3. Acoplamiento de UI Angular a Lógica de Negocio**

Componentes de interfaz realizando transformaciones complejas de datos o cálculos financieros directamente en métodos del ciclo de vida. **Acción:** Desacoplar a Presenters o Signal Stores.

Estrategias de Validación: El Escudo del Desarrollador

Combinando Herramientas Estáticas y Pruebas Automatizadas

FILTRO 1: ANÁLISIS ESTÁTICO

Compilación y Linting Estricto

- **Compilador C# / TypeScript:** Flags de nulabilidad estricta (`strictNullChecks: true`).
- **NetArchTest:** Reglas automatizadas que impiden compilación si Dominio importa Infraestructura.
- **ESLint / Nx Boundary Rules:** Control de dependencias entre librerías.

FILTRO 2: PRUEBAS DINÁMICAS

TDD & Pruebas de Contrato

- **Strict TDD Mode:** Escribir/validar las pruebas unitarias antes o durante la generación del código.
- **Pruebas ACL:** Garantizan que las llamadas al esquema legacy GeneXus respeten los mappings sin alterar tablas.
- **Ejecución Local Rápida:** Verificación inmediata mediante CLI antes del commit

Evolución: Hacia una Arquitectura Agéntica

De la Asistencia Puntual a Pipelines de Migración Estandarizados



Fase 1: Chat Prompting

Asistencia aislada, prompts manuales copiados en el navegador.



Fase 2: Copiloto en IDE

Autocompletado de funciones y sugerencias contextuales.



Fase 3: SDD Agéntico

Agentes orquestados, herramientas CLI, Claim-Check y control HITL.



A continuación: Analizaremos en detalle la arquitectura agéntica real de nuestro proyecto de migración de **Fideicomisos**.

Principios Infranqueables del Sistema Agéntico

Reglas de Integridad, Inmutabilidad Legacy y Gobernanza

1. Zero Hallucination & GeneXus ACL

- **Tech Stack Whitelist:** Prohibido inventar librerías o comandos. Todo elemento debe estar en .
- **GeneXus Read-Only Invariant:** Las tablas heredadas GeneXus son 100% de solo lectura. Prohibido ejecutar DDL o scripts de migración sobre ellas. Toda integración se realiza mediante la capa ACL.
- **Validación Física de Manifiestos:** El orquestador comprueba en disco que el

2. Trade-Off First & Control HITL

- **Protocolo Trade-Off First:** Obligación de presentar al menos 2 alternativas técnicas (Pros, Contras, Deuda Técnica) antes de escribir código.
- **Sin Reintentos Infinitos:** Prohibido entrar en bucles de auto-reintento tras fallas de compilación o pruebas. Se requiere pausa HITL obligatoria.

Hidratación JIT (Lazy Loading) de Contexto

Evitando la Saturación de Tokens (*Token Bloat*) mediante Carga Perezosa por Alcance

EL PROBLEMA VS LA SOLUCIÓN JIT

Carga Masiva vs. Carga Bajo Demanda

- **El Antipatrón de Carga Masiva:** Cargar toda la documentación y reglas del monorepo consume 50,000+ tokens y provoca pérdida de atención (*Lost in the Middle*).
- **La Solución JIT (Just-In-Time):** Carga perezosa de contexto. El sistema hidrata el archivo de memoria únicamente cuando lo

LOS 3 NIVELES DE SCOPE

Ámbitos de Hidratación en AGENTS.md

Nivel 1 — Global Scope:

Prompts ambiguos o de arquitectura general. Carga `global-architect-orchestrator.md` y `global-lessons-learned.md`.

Nivel 2 — Backend Scope (`/backend`):

Operaciones en .NET 10. Carga `backend-lessons-learned.md` + Agente Backend activo (ej. `tactical-ddd-modeler`).

Nivel 3 — Frontend Scope (`/frontend`):

Operaciones en Angular 22. Carga `frontend-lessons-learned.md` + `frontend-ui-architect`.

Catálogo de Agentes Especializados del Proyecto

Estructura Jerárquica de 8 Agentes en 3 Niveles de Dominio

ORQUESTACIÓN GLOBAL

Gobernanza & Estado

- **global-architect-orchestrator:** Gobernador del pipeline SDD, cálculo de DCS y despacho.
- **system-state-analyzer:** Explorador táctico de solo lectura (*Read-Only Scout*) para telemetría.
- **git-commit-expert:** Verificador de build por capa, auditor de LOC y creador de commits en Español.

BACKEND (.NET 10 / DDD)

Dominio & Hexagonal

- **strategic-ddd-architect:** Diseñador técnico backend y creador de listas de tareas tácticas.
- **tactical-ddd-modeler:** Modelador de Aggregates, Value Objects inmutables y Handlers CQRS.
- **acl-legacy-integration-expert:** Constructor de adaptadores EF Core y Capa Anti-Corrupción GeneXus.
- **domain-quality-testing-expert:** Automatizador de pruebas xUnit y Testcontainers.

FRONTEND (ANGULAR 22 / NX)

Signals & UI Adapters

- **frontend-ui-architect:** Diseñador UI y creador de componentes zoneless con Signals.
- **Arquitectura Nx:** Separación estricta entre componentes Smart (Store/State) y Dumb (UI Pura).
- **Zero Zone.js:** Manejo de estado reactivo moderno mediante `signal()`, `computed()` y `httpResource()`.

Matriz de Skills & Gestión de Memoria

Habilidades Inyectadas Dinámicamente y Archivos de Persistencia

MATRIZ DE SKILLS ESPECIALIZADAS

Habilidades Inyectadas por Ámbito

Skill	Ámbito	Propósito
trade-off-analysis	Global	Evaluación de 2+ opciones de diseño
hexagonal-architecture	Backend	Aislamiento Dominio/Aplicación/Infra
ddd-patterns	Backend	Aggregates, Value Objects & Smart Enums
angular-nx-		Signals, Zoneless en Memory

CONOCIMIENTO & MEMORIA PERSISTENTE

Persistencia en knowledge/ y memory/

- Knowledge (knowledge/): Contexto estático verificado:
 - knowledge/employees/ (SLP, Zacatecas, SITTEZ...).
 - knowledge/employees/ (Librerías aprobadas).
 - knowledge/employees/ (Límites de

Algoritmo DCS & Enrutamiento Determinista

Evaluación de Complejidad Estructural y Selección de Rama (Branch A vs. B)

FÓRMULA MATEMÁTICA DCS

Deterministic Complexity Score

$$\text{DCS} = N_{\text{files}} + 3(N_{\text{layers}}) + 5(N_{\text{contexts}}) + 8(S_{\text{schema}})$$

- **N_files:** Archivos estimados a modificar ().
- **N_layers:** Capas impactadas (Dominio, App, Infra/ACL, UI).
- **N_contexts:** Bounded Contexts afectados.
- **S_schema:** Indicador binario de quiebre de esquema de API/DTO (1 o 0).

CRITERIO DE RAMAS & HITL GATE 1

Selección de Rama de Ejecución

Branch B (Fast-Track / Hot-Fix):

Si $\text{DCS} \leq 3$ y $\text{LOC} \leq 100$. Omite fase de diseño pesado y ejecuta implementación directa + commit + QA.

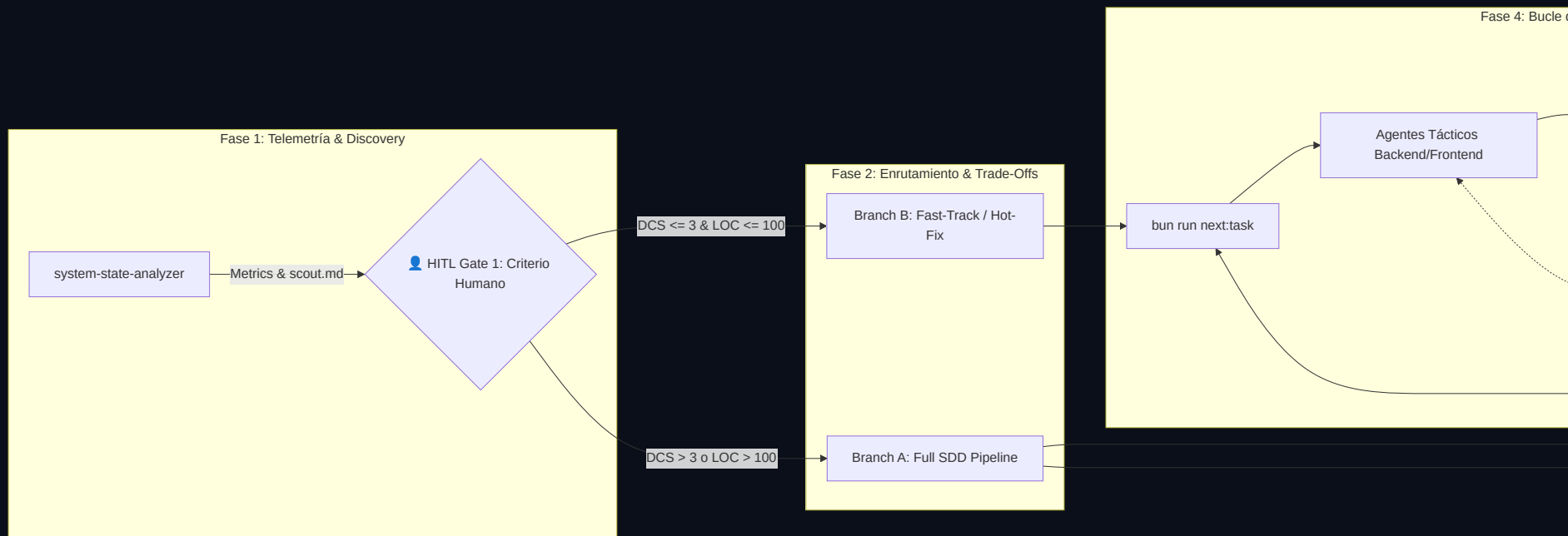
Branch A (Full 5-Phase Pipeline):

Si $\text{DCS} > 3$ o $\text{LOC} > 100$. Ejecuta el pipeline completo de 5 Fases.

🛑 **Gate 1 HITL:** El orquestador TIENE PROHIBIDO enrutar automáticamente. Muestra métricas y espera confirmación humana.

Pipeline SDD de 5 Fases

Flujo Estandarizado de Migración & Puertas HITL



Commit Atómico & Protocolo Stash Guard

Garantizando la Integridad de la Compilación e Aislamiento de Errores

BUCLE DE COMMIT ATÓMICO

Proceso por Tarea Táctica (Fase 4)

1. **Extracción Metadatos:** []
[] entrega [].
2. **Manifiesto de Archivos:** El agente táctico emite [].
3. **Validación de Build:** []
ejecuta [] o [].

STASH GUARD ANTE FALLAS

Aislamiento Mecánico de Errores

- **Fallo de Compilación:** Si []
[] reporta error de build, la tarea NO se commitea.
- **Aislamiento Stash:** Se ejecuta []
[] para enviar los cambios al stash de Git.
- **Traceback en Disco:** Se escribe el error en [].

Herramientas CLI & Puertas de Control HITL

Automatización Mecánica y Control Humano Infranqueable

Comandos CLI Automatizados

- `cli get-task` — Extrae la siguiente tarea lista en el DAG.
- `cli mark-task` — Marca casillas en Markdown tras verificación.
- `cli check-disk` — Comprueba físicamente existencia y datos en disco (> 0 bytes).
- `cli isolate` — Aísla o restaura cambios ante fallas de compilación.

Las 3 Puertas Human-in-the-Loop (HITL)

Gate 1 — Selección de Branch (Fase 2):

El desarrollador aprueba el score DCS y la rama de ejecución (A o B).

Gate 2 — Stash Guard (Fase 4):

Ante fallos de compilación, el desarrollador decide la estrategia de reparación.

Gate 3 — Fallo de QA (Fase 5):

Prohibido reintentos automáticos infinitos. Se solicita clasificación humana (Código, Diseño o Aserto).

El Desarrollador al Mando

"La Inteligencia Artificial magnifica nuestra velocidad de desarrollo,
pero la arquitectura, la calidad y el diseño técnico
son 100% responsabilidad del equipo de ingeniería."

¿Preguntas, comentarios o debate sobre la arquitectura agéntica?

SESIÓN 8 COMPLETADA

LISTO PARA MIGRACIÓN